

Computer Science & IT

Data Structure & Programming



Stack

Lecture No. 03



By- Abhishek Sir

Recap of Previous Lecture



Topic

Expression

Topic

prefix . polish Notation

Topic

postfix : Reverse polish

Topic

precedence and Associativity

Topic

using stack

Topics to be Covered



Topic

Evaluation of postfix expression

Topic

Topic

Topic

Topic



Infix to Postfix using stack

1. Scan input character left to right.
2. Output character left to right.
3. If the input is an operand then print it.
4. If input is the operator and stack is empty then push the operator.
5. If input is the operator having higher precedence than operator on top of stack the perform push Operation (HPU).



Infix to Postfix using stack

6. If the input operator is having lower precedence than operator on the top of stack then pop operators until input operator is having lower precedence than operator on top of stack and push new operator. LPOPU-S
7. Left Associative LPOPU-S
8. Right Associative HPU



Infix to Postfix : a * b / c

* higher precedence than /

Symbol	Stack					Output
a						a
*	*					a
b						ab
/	*	LPOPUS				ab
	/					ab*
c						ab*c
	End of input : pop everything					ab*c/

Left Associative
LPOPUS

Infix to Postfix : a ₁ b ₂ c



$\uparrow_2 >$ Right Associative

Symbol	Stack					Output
a						a
$\uparrow_1$	$\uparrow_1$					a
b	$\uparrow_1$					ab
$\uparrow_2$	$\uparrow_1$	$\uparrow_2$	<u>HPU</u>			ab
c	$\uparrow_1$	$\uparrow_2$				abc
	$\uparrow_1$	$\uparrow_2$	End of INPUT			abc ↑ ↑

Right associative

Higher precedence Rule

HPU

Infix to prefix using stack

1. Scan Right to Left
2. print right to Left
3. if input is operator and stack is empty, push the operator
4. if input is operand print
5. End of input, pop everything and print
6. Input is higher precedence than operator on top of stack: HPU

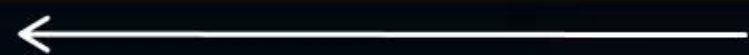
7. Lower precedence : LPOPUS

8. Left Associative : HPU

9. Right Associative : LPOPUS



Infix to prefix: $a + b * c$



$a + \underline{*bc}$
R to Left $+ a * bc$

Symbol	Stack					Output
c						c
*	*					c
b						bc
+	*	Lpopu				bc
	+					*bc
a	+					a*bc
	End of input					+a*bc

Infix to prefix

$a * b - c$

Right to Left

$$\begin{array}{r} a * b - c \\ \hline * a b - c \\ \hline - * a b c \end{array}$$



Symbol	Stack					Output
c						c
-	-					c
b	-					bc
*	-	<u>HPU</u>				bc
	-	*				bc
a	-	*				abc
	pop every thing & print					- * abc



Infix to Prefix : a * b / c

* higher precedence than /

Symbol	Stack					Output
c						c
/	/					c
b	/					bc
*	/	HPU				bc
	/	*				bc
a	/	*				abc
	End of Input pop and print					/*abc

Infix to prefix
Left Associative
HPU
Right Associative
LPOPUS



Topic : Evaluation of Postfix Expression

6 2 3 + - 3 8 2 / + * 2 ^ 3 + Single scan Evaluate the expression

6	2	3	+	-	3	8	2	/	+	*	2	^	3	+			
							2										
		3				8	8	$8/2=4$									
	2	2	$2+3=5$		3	3	3	3	$3+4=7$		2		3				
6	6	6	6	$6-5=1$	1	1	1	1	1	$1*7=7$	7	$7^2=49$	49	$49+3=52$			

Scan Left to right
if input is operand
push the operand

if input is operator
operand₂ = pop();
operand₁ = pop();
Val = operand₁ op operand₂
push(val)

Conversion - stack operator stack

Evaluation stack : operand stack



Topic : Evaluation of Postfix Expression

$$\underline{6\ 2\ 3\ +} - 3\ 8\ 2\ /\ + * 2\ \uparrow\ 3\ +$$

$$\begin{aligned} & \underline{6,\ 2,\ 3\ +} \quad (2+3) \\ = & \underline{6\ 5\ -} \quad (6-5) \\ = & \underline{1,\ 3,\ 8,\ 2\ /\ } \quad (8/2) \\ = & \underline{1\ 3\ 4\ +} \quad (3+4) \end{aligned}$$

$$= \underline{1,\ 7\ *} \quad (1*7)$$

$$= \underline{7\ 2\ \uparrow} \quad (7\uparrow 2)$$

$$= \underline{49,\ 3,\ +} \quad (49+3)$$

$$= 52$$

$$\begin{aligned} & \underline{6,\ 2\ 3\ +} \\ & \quad \downarrow \\ & \underline{6\ +\ 2\ 3\ -} \\ & \underline{-\ 6\ +\ 2\ 3\ ,\ 3\ ,\ 8,\ 2\ /\ } \\ & \underline{-\ 6\ +\ 2\ 3\ ,\ 3\ ,\ 1\ 8\ 2\ +} \\ & \underline{-\ 6\ +\ 2\ 3\ ,\ +\ 3\ 1\ 8\ 2\ *} \end{aligned}$$

$$\begin{aligned} & \underline{* - 6 + 23 + 3 / 82} . 2\ \uparrow \\ + \uparrow & * - 6 + 23 + 3 / 822\ 3 \end{aligned}$$



Topic : Evaluation of Postfix Expression

2,3,*,4,2,2,*,/,+,3,5,+,-

* *

/ +

+ -

if expression evaluates to x then

$x + 10$ is $-1 + 10 = 9$

	2															
	2	4			5											
3	4	4	1		3	8										
2	6	6	6	7	7	7										

$7 - 8 = (-1)$



Topic : Evaluation of Prefix Expression

$+ \uparrow * - 6 + 23 + 3 / 8223$

operand₁ = pop()
operand₂ = pop()

	/		←	+	+		-	*	↑	+
8		3		$\frac{2}{3}$	5	$\frac{6}{5}$	$6-5=1$			
2	$\frac{8}{2}=4$	4	$7(3+4)$	7	7	7	7	$\frac{1*7}{=7}$		
2	2	2	2	2	2	2	2	2	$7^{\uparrow 2}=49$	
3	3	3	3	3	3	3	3	8	3	$\frac{49+3}{=52}$



Topic : Question



#Q. The following postfix expression with single digit operands is evaluated using a stack:

$$823^{\wedge} / 23 * + 51 * -$$

Note that $\wedge$ is the exponentiation operator. The top two elements of the stack after the first $*$ is evaluated are:

☒ (A) 6, 1

(B) 5, 7

(C) 3, 2

(D) 1, 5

$$\boxed{8 \mid 2 \mid 3}^{\wedge} \quad (2^3)$$

$$\boxed{8 \mid 8} / \quad (8/8)$$

$$\boxed{1 \mid 2 \mid 3} * \quad (2*3 = 6)$$

$$\boxed{1 \mid 6}$$

↑ top



Topic : Question

Consider two binary operators ' $\uparrow$ ' and ' $\downarrow$ ' with the precedence of operator $\downarrow$ being lower than that of the operator $\uparrow$. Operator $\uparrow$ is right associative while operator $\downarrow$, is left associative.

$\uparrow$ is exponentiation operator, $a \uparrow b = a^b$

$\downarrow$ is logarithmic operator $a \downarrow b = \log_b a$

The value of the expression $(65536 \downarrow 2 \uparrow 4 \uparrow 2 \downarrow 2)$ is _____.

$\uparrow$ precedence higher

$\uparrow$ - Right Associative

$\downarrow$ Left Associative

$$65536 \downarrow 2 \uparrow \underline{4 \uparrow 2} \downarrow 2$$

$$65536 \downarrow 2 \uparrow 16 \downarrow 2$$

$$65536 \downarrow 65536 \downarrow 2$$

$$\log_{65536} 65536 = 1$$

$$= 1 \downarrow 2 = \log_2 1 = 0$$



Topic : Question



Q Which of the following expressions evaluates to the largest number?

(A) The postfix expression $2\ 3\ +\ 5\ * \ 7\ -$ $\rightarrow 5, 5* = 25, 7 - = 18$

(B) The prefix expression $+ \ * \ - \ 2\ 3\ 5\ 7$ $\rightarrow - \underline{2\ 3}\ 5\ 7 = * - 1, 5, 7 = +, - 5, 7 = - 5 + 7 = 2$

(C) The infix expression $(2 + 3) * (5 - 7)$ $\rightarrow 5 * - 2 = -10$

(D) The infix expression $2 + 3 * 5 - 7$ $\rightarrow 2 + 15 - 7 = 10$



Topic : Question



A function f defined on stacks of integers satisfies the following properties. $f(\phi) = 0$ and $f(\text{push}(S, i)) = \max(f(S), 0) + i$ for all stacks S and integers i .

If a stack S contains the integers 2, -3, 2, -1, 2 in order from bottom to top, what is $f(S)$?

(A) 6

(B) 4

(C) 3

(D) 2

$$f\left(\begin{array}{|c|} \hline 2 \\ \hline -1 \\ \hline 2 \\ \hline -3 \\ \hline 2 \\ \hline \end{array}\right)$$

$\Downarrow$

$$f\left(\text{push}\left(\begin{array}{|c|} \hline -1 \\ \hline 2 \\ \hline -3 \\ \hline 2 \\ \hline \end{array}, 2\right)\right)$$

$$= \max\left(f\left(\begin{array}{|c|} \hline -1 \\ \hline 2 \\ \hline -3 \\ \hline 2 \\ \hline \end{array}\right), 0\right) + 2$$



Topic : Question



$$\textcircled{3} \quad f\left(\begin{array}{|c|c|c|c|c|} \hline 2 & -3 & 2 & -1 & 2 \\ \hline \end{array}\right)$$

A function f defined on stacks of integers satisfies the following properties. $f(\phi) = 0$ and $f(\text{push}(S, i)) = \max(f(S), 0) + i$ for all stacks S and integers i .

If a stack S contains the integers 2, -3, 2, -1, 2 in order from bottom to top, what is $f(S)$?

(A) 6

(C) 3 ✓

(B) 4

(D) 2

2005 (IT)

$$\begin{aligned} & \text{push}(\phi, 2) \\ &= \max(f(\phi), 0) + 2 \\ & \quad (0, 0) \end{aligned}$$

$$\begin{aligned} & f(\text{push}(\begin{array}{|c|c|c|c|} \hline 2 & -3 & 2 & -1 \\ \hline \end{array}, 2)) \\ &= \max\left(f\left(\begin{array}{|c|c|c|} \hline 2 & -3 & 2 \\ \hline \end{array}\right), 0\right) + 2 \\ &= \max\left(f(\text{push}(\begin{array}{|c|c|} \hline 2 & -3 \\ \hline \end{array}, 2)), 0\right) + 2 \\ &= \max\left(f\left(\begin{array}{|c|} \hline 2 \\ \hline \end{array}\right), 0\right) - 3 \\ &= \max(2, 0) - 3 \\ &= 2 - 3 \\ &= -1 \end{aligned}$$

Topic : Question MSQ

#Q. Consider the post fix expression $5\ y\ *\ 3\ 4\ +\ 5\ y\ **\ +$

$$5y, 3, 4 +$$

$$5y, 7, 5y * = 5y, 35 + = 40y$$

Which of the following is correct

(a) If expression evaluated to 80 then y is 2

$$40y = 80 \Rightarrow y = 2$$

$$5, 2 * = 10, 3, 4 + = 10, 7, 5, 2 * = 10, 7, 10 * = 10, 70 + = 80$$

(b) If expression evaluated to 160 then y is 4

$$40y = 160 \Rightarrow y = 4$$

$$5, 4 * = 20, 3, 4 + = 20, 7, 5, 4 * = 20, 7, 20 * = 20, 140 = 160$$

(c) If expression evaluated to 200 then y is 5

$$40y = 200 \Rightarrow y = 5$$

(d) If expression evaluated to 160 then y is 5

$$5, 5 * = 25, 3, 4 + = 25, 7, 5, 5 * = 25, 7, 25 * = 25, 175 = 200$$

$$= 25, 7, 25 * = 25, 175 = 200$$

#Q. Let S_1 and S_2 be two stacks. S_1 has capacity of 4 elements. S_2 has capacity of 2 elements. S_1 already has 4 elements: 100, 200, 300, and 400, whereas S_2 is empty, as shown below.

400 (Top)	300
300	400
200 300 400	Stack S_1
100	
Stack S_1	

output sequence (A) 200, 300, 400, 100 ✓

push to S_2 . push to S_2 GO push to S_1 GO, push to S_1 GO
 ↓
 200 GO

Only the following three operations are available:

PushTo S_2 : Pop the top element from S_1 and push it on S_2 .

PushTo S_1 : Pop the top element from S_2 and push it on S_1 .

Generate Output: Pop the top element from S_1 and output it to the user.

Note that the pop operation is not allowed on an empty stack and the push operation is not allowed on a full stack.

#Q. Let S_1 and S_2 be two stacks. S_1 has capacity of 4 elements. S_2 has capacity of 2 elements. S_1 already has 4 elements: 100, 200, 300. and 400. whereas S_2 is empty, as shown below.

400 (Top)	
300	300
200	Stack S_1
100	
Stack S_1	

✓ (B) 400, 200, 100, 300
GO push to S_2 GO, GO,
 200
 push to S_1 GO

Only the following three operations are available:

PushTo S_2 : Pop the top element from S_1 and push it on S_2 .

PushTo S_1 : Pop the top element from S_2 and push it on S_1 .

Generate Output: Pop the top element from S_1 and output it to the user.

Note that the pop operation is not allowed on an empty stack and the push operation is not allowed on a full stack.

#Q. Let S_1 and S_2 be two stacks. S_1 has capacity of 4 elements. S_2 has capacity of 2 elements. S_1 already has 4 elements: 100, 200, 300, and 400, whereas S_2 is empty, as shown below.

400 (Top)	
300	
200	Stack S_1
100	
Stack S_1	

(c) 100, 200, 400, 300

Not possible

Only the following three operations are available:

PushTo S_2 : Pop the top element from S_1 and push it on S_2 .

PushTo S_1 : Pop the top element from S_2 and push it on S_1 .

Generate Output: Pop the top element from S_1 and output it to the user.

Note that the pop operation is not allowed on an empty stack and the push operation is not allowed on a full stack.

#Q. Let S_1 and S_2 be two stacks. S_1 has capacity of 4 elements. S_2 has capacity of 2 elements. S_1 already has 4 elements: 100, 200, 300, and 400, whereas S_2 is empty, as shown below.

push to S_2 GO GO, push to S_1 GO, GO

400 (Top)	
300	
200 400	400
100	Stack S_1
Stack S_1	

~~(D)~~ 300, 200, 400, 100

Only the following three operations are available:

PushTo S_2 : Pop the top element from S_1 and push it on S_2 .

PushTo S_1 : Pop the top element from S_2 and push it on S_1 .

Generate Output: Pop the top element from S_1 and output it to the user.

Note that the pop operation is not allowed on an empty stack and the push operation is not allowed on a full stack.

Which of the following output sequences can be generated by using the above operations?

- A** 200, 300, 400, 100 ✓
- B** 400, 200, 100, 300 ✓
- C** 100, 200, 400, 300
- D** 300, 200, 400, 100 ✓

Queue

THANK - YOU